



When to Use Microservices

Deciding If Microservices Are Right for You



Why Microservices Aren't Always Best



Flexibility Comes with Complexity

Microservices increase operational and architectural overhead



Not a Universal Solution

Many projects gain little from microservices adoption



Evaluate Before Adopting

Assess team size, domain, and scaling needs first



Key Factors in the Decision

Team Size Drives Architecture

Large, independent teams benefit from microservices.

Domain Complexity Matters

Complex business logic may require service separation.

Scaling and Technology Needs

Independent scaling and diverse tech stacks favor microservices.

Release Cycle Flexibility

Frequent, independent releases are easier with microservices.





When Microservices Make Sense

Enables Large Teams to Work in Parallel

Reduces code conflicts and boosts team autonomy

Handles Complex Business Domains

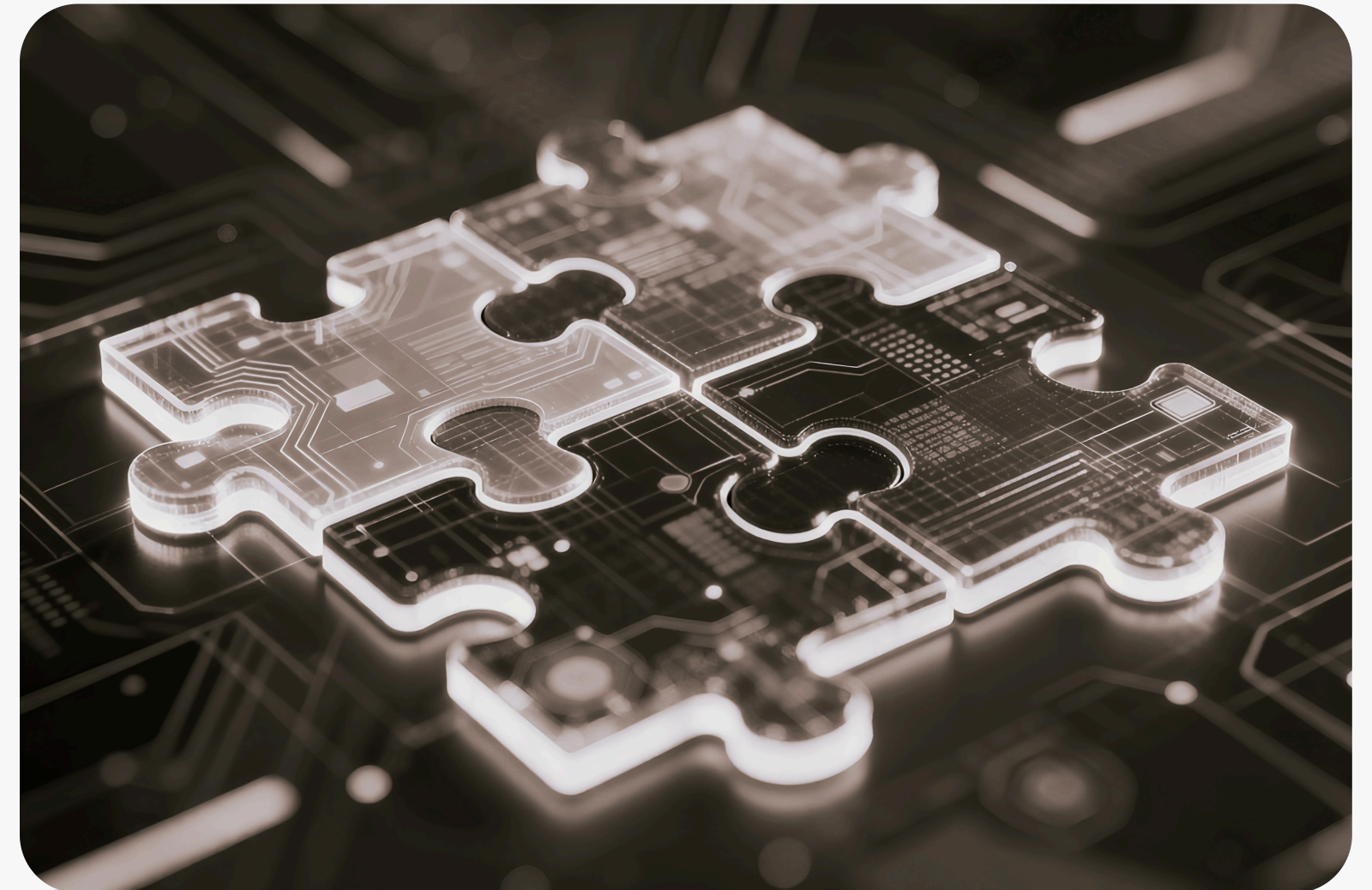
Manages multiple contexts and business rules efficiently

Supports Independent Scaling

Scale only the services that need extra resources

Allows Technology Diversity & Fast Releases

Use the best tech for each service and release features independently





When to Avoid Microservices

Unclear Requirements in New Products

Frequent changes make microservices risky and costly

Small Teams Face High Overhead

Managing multiple services strains limited resources

Simple Domains Don't Need Complexity

Minimal business logic fits better in a monolith

DevOps Immaturity or Tight Deadlines

Lack of automation and time pressure hinder microservices success





A Smarter Path: Evolve, Don't Revolt

Begin with a Modular Monolith

Build new projects as a single, well-structured application

Extract Microservices Gradually

Split off services only when scaling or complexity demands it

Embrace Hybrid Architectures

Combine monolith and microservices for optimal flexibility



Checklist Before You Decide

Assess Team and Independence

Large, autonomous teams benefit most from microservices.

Evaluate Domain Complexity

Complex, well-understood domains are better suited.

Check Scaling and Tech Needs

Independent scaling or diverse tech stacks favor microservices.

Review DevOps Maturity

A mature DevOps pipeline is essential for success.

Start Simple if Unsure

If most answers are no, begin with a monolith.



Key Takeaways

Best for Large, Fast-Moving Teams

Microservices empower big teams to work in parallel.

Start Simple, Grow as Needed

Begin with a monolith and evolve architecture over time.

DevOps Maturity is Critical

Robust CI/CD and monitoring are essential for success.

Hybrid Approaches Work Well

Mix monolith and microservices for optimal results.